ULEAM

UNIVERSIDAD LAICA
ELOY ALFARO DE MANABÍ

PEA: App de superhéroes

Desarrollo de una aplicación Android
con Kotlin, Jetpack Compose y Material 3

ESTUDIANTE

Carlos Luis Ortiz García

CARRERA

Ingeniería en Software

ASIGNATURA

Aplicaciones Móviles

DOCENTE

Ing. Edgardo Panchana

FECHA

18 de octubre de 2026

Informe técnico y evidencias de la implementación

Objetivo y requisitos

Correspondencia entre la consigna y el proyecto entregado.

1. Objetivo

Implementar una aplicación Android que presente seis superhéroes mediante una lista desplazable. La práctica permite aplicar composición de interfaces, recursos externos, reutilización de componentes y personalización visual con Material 3.

2. Cumplimiento del ejercicio

Requisito	Evidencia en el proyecto
Base Android y API mínima 24	ComponentActivity, módulo app y minSdk = 24.
Modelo y datos locales	Hero.kt y HeroesDataSource.kt dentro de model; seis registros.
Textos, imágenes e icono	strings.xml, seis vectores android_superhero e iconos mipmap oficiales.
Fuente Cabin y Material 3	Cabin Regular/Bold; Color.kt, Type.kt, Shape.kt y Theme.kt.
Tarjeta de héroe	Elevación 2 dp; forma 16 dp; padding 16 dp; imagen 72 dp y radio 8 dp.
Lista y barra superior	LazyColumn; CenterAlignedTopAppBar centrada dentro de Scaffold.
Claro, oscuro y bordes	Tema según el sistema; padding de Scaffold y ajuste de barras.

Interpretación de la altura: los 72 dp corresponden al contenido mínimo de la fila y al tamaño de la ilustración. El padding se agrega por fuera del contenido; la tarjeta puede crecer si el nombre o la descripción ocupan más líneas.

Se mantienen en inglés los nombres y las descripciones del conjunto de datos del ejercicio para conservar la referencia visual del codelab.

Diseño e implementación

Separación de datos, componentes y sistema visual.

3. Organización del código

Datos: Hero almacena tres identificadores de recursos: nombre, descripción e imagen. Las anotaciones @StringRes y @DrawableRes hacen explícito el tipo de recurso esperado. HeroesRepository reúne la lista fija y evita repetir datos en la interfaz.

Presentación: MainActivity configura Compose; SuperheroesApp coordina Scaffold y la barra superior. HeroesList crea los elementos bajo demanda con LazyColumn. HeroListItem concentra la tarjeta, la columna de textos y la ilustración.

```
SuperheroesTheme {
    SuperheroesApp(heroes = HeroesRepository.heroes)
}

Scaffold(topBar = { SuperheroesTopAppBar() }) { padding ->
    HeroesList(heroes = heroes, contentPadding = padding)
}
```

4. Decisiones técnicas

La pantalla recibe los héroes como parámetro, lo que facilita su reutilización y las pruebas. Cada elemento usa nameRes como clave estable. Los textos se obtienen con stringResource y las imágenes con painterResource; todos los recursos funcionan sin conexión.

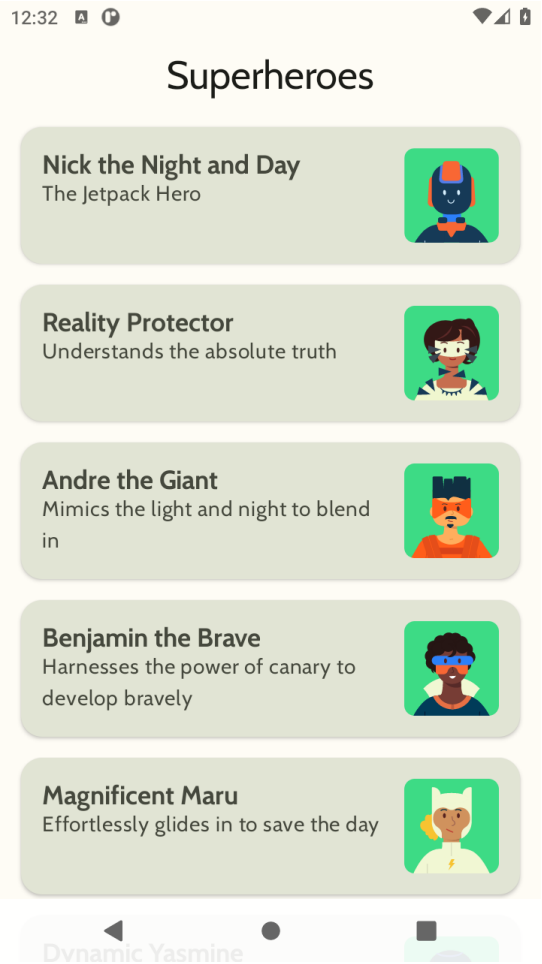
Las tarjetas conservan la animación de entrada de la solución oficial: aparición de la lista y desplazamiento vertical escalonado. Las imágenes se recortan con esquinas de 8 dp. Su descripción accesible es nula porque el nombre del héroe ya está presente como texto y la ilustración es decorativa.

El tema usa paletas fijas clara y oscura, sin color dinámico por defecto. Cabin define los títulos y el cuerpo. WindowCompat permite dibujar de borde a borde; el Scaffold aporta los márgenes necesarios para que el contenido no invada las barras.

Alcance: la aplicación es un catálogo de datos locales. La actividad no exige autenticación, almacenamiento remoto ni una API de superhéroes.

Interfaz y evidencias

Renderizado de la aplicación y comportamiento de la lista de héroes.



Tema claro



Tema oscuro

Las capturas muestran la barra centrada, las tarjetas con tipografía Cabin y las ilustraciones oficiales. El tema sigue la configuración del sistema; la disposición mantiene la misma estructura en ambos modos.

5. Comportamiento esperado

Al abrir la app se muestra Superheroes y el primer registro, Nick the Night and Day. Al desplazar hacia arriba se llega a Dynamic Yasmine. Las descripciones largas aumentan la altura de la tarjeta sin imponer un recorte fijo.

Para revisar el modo oscuro se cambia el tema del dispositivo y se vuelve a la app. Los fondos, textos y superficies adoptan la paleta oscura; el contenido de los héroes se conserva.

Validación y ejecución

Resultados observados y pasos para reproducir la entrega.

6. Comprobaciones realizadas

Comprobación	Resultado
Compilación y APK	Correcta en Windows y GitHub Actions. APK de depuración generado (11,92 MB).
Android Lint	Correcto: 0 errores y 12 advertencias. Se conserva la base visual y las versiones verificadas.
Pruebas instrumentadas	2 aprobadas, 0 fallidas. Tema claro: título y último héroe; tema oscuro: renderizado y desplazamiento.
Ejecución y capturas	Correcta en emulador Android 11/API 30 de GitHub Actions. Capturas de ambos temas y de la lista desplazada.
Versionado	Proyecto publicado en main; código verificado: bde9d5e. Documentación, evidencias y APK incluidos en la entrega.

La validación de ejecución se realizó en Android 11/API 30; API 24 y otros tamaños requieren dispositivos adicionales. Lint local conserva 12 advertencias por targetSdk 35, versiones de bibliotecas, tamaño nominal de vectores oficiales y etiqueta redundante del manifiesto. El APK es de depuración para evaluación.

7. Reproducción en Android Studio

1. Clonar el repositorio o descomprimir el proyecto y abrir la carpeta SuperheroesULEAM.

2. Configurar el JDK de Gradle. La entrega se verifica con JBR 25, Gradle 9.6.0, AGP 9.4.0 y el plugin Compose 2.2.10.

3. Instalar SDK Platform 37, sincronizar Gradle y conectar un dispositivo Android API 24 o superior. local.properties se genera con la ruta del SDK de cada equipo.

4. Ejecutar app, desplazar hasta el sexto héroe y cambiar el tema del sistema para revisar ambas paletas.

```
.\gradlew.bat assembleDebug lintDebug assembleDebugAndroidTest
.\gradlew.bat connectedDebugAndroidTest
```

El segundo comando requiere un emulador o dispositivo conectado. El APK de depuración se incluye en entrega/SuperheroesULEAM-debug.apk; el código fuente y los recursos están en el mismo repositorio.

Repositorio y conclusiones

Trazabilidad del repositorio y cierre de la práctica.

8. Control de versiones y entrega

Se creó el repositorio y se realizó un primer commit del proyecto Android. Los siguientes cambios incorporan la documentación, el APK y las evidencias. Los archivos temporales, cachés, rutas locales y claves de firma no forman parte del código versionado.

Repositorio:
github.com/CLOG-U/uleam-superheroes-compose

```
git clone https://github.com/CLOG-U/uleam-superheroes-compose.git
cd uleam-superheroes-compose
git log --oneline
```

9. Conclusiones

La práctica integra datos locales con una interfaz declarativa. La separación entre modelo, repositorio, componentes y tema facilita localizar cada responsabilidad. LazyColumn ofrece una lista desplazable y MaterialTheme mantiene coherencia de color, forma y tipografía.

El tratamiento de padding, alturas mínimas y barras del sistema evita que una medida rígida o el borde de la pantalla oculte información. Las pruebas incluidas verifican la presencia del encabezado y el acceso al último héroe en ambos temas.